Licence 3 Informatique

Analyse d'identité d'auteurs et détection de plagiat

Auteurs :
Morel E. TALON
Lionel NAHOU
Roberto HOUNGBO
Sèna GOUBALAN

Encadrant TP :
Gregory BONNET

Année universitaire 2025–2026

Table des matières

Introduction **3**

1 État de l'art **5**
1.1 Détection de plagiat et attribution d'auteurs 5
1.2 Représentation des textes et mesures de similarité 5

2 Organisation du Projet **6**
2.1 Répartition des tâches 6
2.2 Architecture globale du projet 6
2.2.1 Structure des modules 6
2.2.2 Interaction des modules 7

3 Éléments techniques **8**
3.1 Corpus et étiquetage 8
3.2 Pré-traitement des données 8
3.3 Mesures de similarité implémentées 9
3.3.1 Similarité cosinus TF-IDF 9
3.3.2 Similarité de Jaccard 9
3.3.3 Similarité de Dice 9
3.3.4 Jaccard sur entités nommées (NER) 9
3.3.5 Similarité sémantique Word2Vec 9
3.4 Classificateurs et modèles d'apprentissage 10
3.5 Stratégie d'évaluation 10
3.5.1 Comparaison des modèles 10
3.5.2 Étude d'ablation 10
3.5.3 Grille complète features x modèles 10
3.5.4 Validation croisée 10
3.5.5 Analyse SHAP 10
3.6 Choix technologiques 11

4 Expérimentation **12**
4.1 Protocole expérimental 12
4.1.1 Partitionnement des données 12
4.1.2 Métriques d'évaluation 12
4.1.3 Déroulement des expériences 12
4.2 Résultats des mesures de similarité 12
4.2.1 Comparaison des classificateurs sur toutes les features 12
4.2.2 Pouvoir discriminant des features individuelles 13
4.3 Performance des classificateurs 13
4.3.1 Grille exhaustive : 635 configurations 13
4.3.2 Étude d'ablation 14

4.3.3 Matrice de confusion du meilleur modèle 16
4.3.4 Analyse SHAP 16
4.4 Analyse et discussion des résultats 17
4.4.1 Complémentarité des features sélectionnées 17
4.4.2 Limites identifiées 17

Conclusion **18**

Introduction

Contexte

Le travail en groupe sur un projet offre une approche collaborative qui permet de tirer parti des compétences individuelles, de favoriser la communication et la résolution de problèmes, de répartir efficacement les tâches et de renforcer le sentiment d'appartenance à l'équipe. C'est dans cette logique qu'il nous a été proposé plusieurs sujets afin de travailler en groupe de quatre (4) étudiants. Le but étant de mettre en pratique nos connaissances acquises tout au long de l'année en traitement automatique du langage pour réaliser une application fonctionnelle de bout en bout. Parmi les sujets proposés, nous avons choisi celui de "Détection de plagiat et analyse d'identité d'auteurs". Cette problématique est particulièrement pertinente dans le contexte académique où l'explosion des documents numériques rend indispensable l'automatisation de la vérification d'intégrité des travaux étudiants.

Problématique

Un contrôle manuel des documents ne suffit plus pour détecter efficacement le plagiat ou attribuer un texte à un auteur. Il devient nécessaire de disposer d'outils capables de comparer automatiquement des documents textuels et de quantifier leur degré de similarité, malgré les reformulations et variantes linguistiques. La question centrale de ce projet est alors la suivante : quelles représentations des textes et quelles mesures de similarité permettent de distinguer au mieux les documents originaux des documents suspects et d'aider à l'identification des auteurs ?

Objectifs

L'objectif principal de ce projet est de mettre en œuvre une chaîne de traitement automatique permettant d'analyser le contenu textuel de documents et d'en évaluer la similarité, dans le but de détecter d'éventuels cas de plagiat et d'étudier l'identité des auteurs. Plus précisément, il s'agit :

1. de constituer ou de sélectionner un corpus de textes adapté aux expériences ;
2. de prétraiter ces textes afin d'obtenir une représentation exploitable par des algorithmes (tokenisation, filtrage, normalisation, etc.) ;
3. de tester plusieurs modes de représentation des documents et plusieurs mesures de similarité ;
4. de mettre en œuvre un ou plusieurs classificateurs supervisés pour décider si deux documents sont similaires ou non, ou pour attribuer un texte à un auteur ;
5. d'évaluer les performances de ces méthodes à l'aide de métriques standard et de comparer les approches entre elles.

Organisation

Ce rapport est structuré en quatre chapitres. Nous commençons, au chapitre 1, par présenter l'état de l'art autour du *BlocksWorld* afin de situer le cadre conceptuel du projet. Le chapitre 2 décrit ensuite l'organisation générale de notre travail et la manière dont les différents modules ont été structurés. Sur cette base, le chapitre 3 détaille les éléments techniques que nous avons développés pour modéliser le monde des blocs, planifier des actions et résoudre les problèmes associés. Enfin, le chapitre 4 illustre l'ensemble à travers les expérimentations réalisées et les résultats obtenus.

Chapitre 1

État de l'art

Ce chapitre présente les concepts clés de la détection de plagiat et de l'attribution d'auteurs, ainsi que les méthodes de représentation textuelle et de mesure de similarité utilisées dans la littérature.

1.1 Détection de plagiat et attribution d'auteurs

La détection de plagiat vise à identifier l'utilisation non autorisée de contenus existants. Elle distingue plusieurs types de plagiat : copie mot à mot, paraphrase (reformulation du texte source), traduction non signalée, et auto-plagiat où un auteur réutilise ses propres travaux sans citation appropriée. Dans le contexte académique, où l'intégrité scientifique est primordiale, ces pratiques compromettent la validité des évaluations et nécessitent des outils de détection automatisés.

L'attribution d'auteurs (authorship attribution) cherche à déterminer l'auteur probable d'un texte anonyme ou contesté. Elle repose sur l'analyse de caractéristiques stylistiques : fréquence d'utilisation de certaines expressions, longueur moyenne des phrases, ponctuation spécifique, choix lexicaux récurrents, ou patterns syntaxiques. Ces « signatures d'auteur » permettent de distinguer les styles d'écriture et de détecter des ruptures stylistiques dans un document, signe potentiel de collage de sources multiples.

Ces deux problématiques sont étroitement liées : la détection de plagiat nécessite souvent une analyse stylistique, tandis que l'attribution d'auteurs peut révéler des emprunts non déclarés par comparaison avec un corpus de référence.

1.2 Représentation des textes et mesures de similarité

Pour rendre les textes comparables par des algorithmes, il faut d'abord les représenter sous forme numérique. Le modèle du *sac de mots* (Bag-of-Words) transforme un document en vecteur où chaque dimension correspond à un mot du vocabulaire, avec sa fréquence d'apparition. La pondération TF-IDF améliore ce modèle en diminuant l'importance des mots trop fréquents (articles, prépositions) tout en valorisant les termes discriminants.

Les n-grammes capturent des informations locales sur la structure du texte, utiles pour détecter des emprunts partiels. Les embeddings (Word2Vec, BERT) produisent des vecteurs continus tenant compte des relations sémantiques, mais sont plus coûteux en calcul.

Une fois vectorisés, les documents sont comparés via des mesures de similarité : la similarité cosinus mesure l'angle entre deux vecteurs (insensible à leur magnitude), l'indice de Jaccard compare les ensembles de mots ou n-grammes, et la distance de Levenshtein quantifie les opérations d'édition (insertion, suppression, substitution) nécessaires pour transformer un texte.

Chapitre 2

Organisation du Projet

Ce chapitre présente la répartition des tâches et l'architecture modulaire du système développé en Python avec scikit-learn, NLTK et spaCy.

2.1 Répartition des tâches

Le projet a été réalisé par un groupe de quatre étudiants. Dès le début, nous avons choisi une méthode de travail collaborative permettant à chacun de comprendre l'ensemble du code et de maîtriser toutes les étapes du projet. Notre approche s'est déroulée en deux phases complémentaires.

Dans un premier temps, chacun de nous a réalisé indépendamment l'implémentation des principaux modules (prétraitement, extraction de caractéristiques, mesures de similarité, classification). Cette organisation nous a permis d'explorer différentes approches techniques, de comparer nos solutions et de retenir, pour chaque partie, la version la plus claire, la plus structurée ou la plus efficace. Ces séances de regroupement nous ont permis de consolider un code commun et d'assurer une compréhension partagée de tous les modules développés.

Dans un second temps, à mesure que le projet devenait plus technique et plus volumineux, nous avons mis en place une répartition plus précise des tâches. Chaque membre de l'équipe s'est vu attribuer des missions spécifiques pour avancer plus rapidement.

Ces tâches étaient associées à des deadlines régulières lors de réunions hebdomadaires afin de maintenir une progression continue. À chaque étape importante, nous synchronisions notre travail via Git pour vérifier la cohérence du projet global, la bonne intégration des modules et le respect des bonnes pratiques de développement.

2.2 Architecture globale du projet

2.2.1 Structure des modules

Le projet est organisé en quatre modules Python dans le répertoire `src/` :

— `preprocessing.py` : Nettoyage et normalisation des textes. La classe `TextPreprocessor` applique successivement la normalisation Unicode, la suppression des URLs,e-mails, etc, la tokenisation (NLTK), le filtrage des mots vides et la lemmatisation optionnelle (spaCy). Supporte l'anglais et le français. La fonction `load_document()` gère la lecture des fichiers texte avec repli sur l'encodage Latin-1 en cas d'échec UTF-8.

— `feature_extraction.py` : Extraction des mesures de similarité entre paires de documents. La classe `FeatureExtractor` calcule sept features : similarité cosinus TF-IDF, Jaccard sur caractères uni-grammes, Dice sur tokens, Jaccard sur entités nommées

(toutes classes, PERSON, ORG via spaCy), et similarité sémantique Word2Vec (Gensim). La fonction `extract_features_from_pairs()` parcourt `labels.csv` et retourne la matrice X, le vecteur y et les noms des features.

— `models.py` : Gestion des classificateurs supervisés. La classe `PlagiarismClassifier` encapsule cinq modèles scikit-learn (RandomForest, GradientBoosting, NaiveBayes, SVM, LogisticRegression) avec encodage des labels (`LabelEncoder`), sauvegarde/chargement via `joblib` et métadonnées JSON.

— `evaluation.py` : Orchestration de l'évaluation complète. Contient : `compare_models` (comparaison des 5 classificateurs), `ablation_study` (127 sous-ensembles de features), `full_grid_search` (635 combinaisons features × modèles), `cross_validate_model` (validation croisée stratifiée 5 plis), `plot_confusion_matrix` et `shap_analysis`. Génère les graphiques dans `data/processed/`.

— `test_quick.py` : Script de test rapide du pipeline complet : extraction des features depuis `labels.csv`, split train/test, entraînement RandomForest, affichage de l'accuracy.

2.2.2 Interaction des modules

Le point d'entrée est `evaluation.py` (ou `test_quick.py` pour un test rapide). Ces scripts lisent `data/labels.csv` qui recense les 95 paires de documents (colonnes `doc_source`, `doc_suspect`, `label`) et localisent les fichiers texte dans `data-plagiarism/`.

`feature_extraction.py` charge chaque paire via `load_document()` de `preprocessing.py`, puis calcule les sept mesures de similarité pour produire la matrice $X \in R^{95\times7}$ et le vecteur de labels y. `models.py` utilise ces données pour entraîner et prédire avec `PlagiarismClassifier`. Enfin, `evaluation.py` orchestre l'ensemble de l'analyse et sauvegarde les résultats dans `data/processed/`.

La communication entre modules s'effectue via des DataFrames Pandas (liste des paires) et des tableaux NumPy (matrice de features, labels encodés).

Chapitre 3

Éléments techniques

Ce chapitre détaille les choix techniques, le corpus utilisé et l'implémentation des principaux composants du système de détection de plagiat.

3.1 Corpus et étiquetage

Le corpus utilisé contient des paires de documents (original, suspect) rédigés en anglais, organisés par *tâches* (`taska`, `taskb`, `taskc`, `taskd`).

Chaque paire est étiquetée selon le type de plagiat :

— `non` : aucun plagiat ;
— `light` : reformulation légère ;
— `cut` : copie partielle avec suppressions ou réarrangements ;
— `heavy` : copie quasi-intégrale.

Le jeu de données final comprend **95 paires**, réparties comme suit :

Classe	Nombre de paires
`non`	38
`heavy`	19
`light`	19
`cut`	19
Total	95

TABLE 3.1 – Distribution des classes dans le corpus

La classe `non` est légèrement surreprésentée (40 %) par rapport aux classes de plagiat (20 % chacune), ce qui constitue un déséquilibre modéré dont il faut tenir compte lors de l'évaluation.

3.2 Pré-traitement des données

Le pré-traitement est géré par la classe `TextPreprocessor` (`preprocessing.py`). Le pipeline appliqué à chaque document est le suivant :

1. **Normalisation Unicode** : uniformise les caractères accentués et les formes composées ;
2. **Mise en minuscules** ;
3. **Suppression** des URLs, adresses e-mail, chiffres et caractères spéciaux ;

4. **Tokenisation** via NLTK (`word_tokenize`) ;
5. **Suppression des mots vides** (*stop words*) depuis les listes NLTK ;
6. **Lemmatisation** (optionnelle) via spaCy.

La lemmatisation est désactivée lors de l'extraction des features afin de conserver les formes de surface, jugées plus pertinentes pour les mesures de similarité lexicale.

3.3 Mesures de similarité implémentées

Sept mesures sont calculées pour chaque paire de documents par la classe `FeatureExtractor` (`feature_extraction.py`).

3.3.1 Similarité cosinus TF-IDF

Les deux documents sont vectorisés avec `TfidfVectorizer` (scikit-learn), puis la similarité cosinus est calculée entre les deux vecteurs. Cette mesure est sensible à la fréquence et à la rareté des termes.

3.3.2 Similarité de Jaccard

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

Calculée sur des caractères uni-grammes (`n=1`), elle mesure le chevauchement brut de l'ensemble des caractères distincts. Sensible aux substitutions lexicales légères.

3.3.3 Similarité de Dice

$$D(A, B) = \frac{2\,|A \cap B|}{|A| + |B|}$$

Calculée sur les tokens de mots. Similaire à Jaccard mais accorde plus de poids aux éléments communs.

3.3.4 Jaccard sur entités nommées (NER)

Après reconnaissance des entités nommées avec spaCy (`en_core_web_sm`), on calcule la similarité de Jaccard sur :

— `ner_jaccard` : toutes les entités nommées ;
— `ner_jaccard_person` : entités de type `PERSON` ;
— `ner_jaccard_org` : entités de type `ORG`.

Ces mesures captent la réutilisation de références spécifiques (personnes, organisations) même en cas de reformulation du reste du texte.

3.3.5 Similarité sémantique Word2Vec

Un modèle Word2Vec (Gensim, `vector_size=50`) est entraîné sur les tokens des deux documents. Chaque document est représenté par le vecteur moyen de ses tokens, et la similarité cosinus entre ces deux vecteurs est calculée. Cette mesure capture la proximité sémantique au-delà du chevauchement lexical exact.

3.4 Classificateurs et modèles d'apprentissage

Cinq classificateurs supervisés sont entraînés et comparés, tous issus de scikit-learn :

Modèle	**Abréviation**	**Paramètres principaux**
Random Forest	RF	200 arbres, profondeur max 15
Gradient Boosting	GB	100 estimateurs, taux 0.1
Naive Bayes gaussien	NB	—
SVM à noyau RBF	SVM	$C = 1.0$, probabilités activées
Régression logistique	LR	$C = 1.0$, max 1000 itérations

Table 3.2 – Classificateurs évalués

L'encodage des labels (`non`, `light`, `cut`, `heavy`) vers des entiers est géré par `LabelEncoder` afin de rester compatible avec l'ensemble des classificateurs.

3.5 Stratégie d'évaluation

3.5.1 Comparaison des modèles

Les modèles sont évalués sur un jeu de test fixe (20 % des données, stratifié) avec les métriques suivantes : accuracy, précision macro, rappel macro et F1-score macro. Le F1 macro est la métrique principale car il pénalise les modèles qui ignorent les classes minoritaires.

3.5.2 Étude d'ablation

Pour identifier la contribution de chaque mesure de similarité, une étude d'ablation exhaustive est conduite : le meilleur modèle trouvé suite à la comparaison, est entraîné sur chaque sous-ensemble non vide des 7 features ($2^7 - 1 = 127$ combinaisons), et le F1 macro est mesuré pour chacune.

3.5.3 Grille complète features x modèles

La grille complète croise les 127 sous-ensembles de features avec les 5 classificateurs, soit **635 configurations** entraînées et évaluées. La meilleure combinaison identifiée est : RF avec {`jaccard_sim`, `ner_jaccard`, `semantic_sim`} ($F_1 = 0.69$).

3.5.4 Validation croisée

Une validation croisée stratifiée à 5 plis (`StratifiedKFold`) est appliquée au meilleur modèle afin d'estimer la variabilité des performances et de détecter un éventuel surapprentissage.

3.5.5 Analyse SHAP

L'importance des features est interprétée via SHAP (*SHapley Additive exPlanations*) avec un `TreeExplainer` pour Random Forest. Les valeurs SHAP quantifient la contribution marginale de chaque feature à chaque prédiction, par classe.

3.6 Choix technologiques

Bibliothèque	**Version**	**Usage**
Python	3.x	Langage principal
scikit-learn	1.8.0	Classificateurs, vectorisation TF-IDF, métriques
spaCy	3.8.11	NER, lemmatisation, tokenisation
Gensim	4.4.0	Word2Vec (similarité sémantique)
SHAP	0.51.0	Interprétabilité des modèles
NLTK	3.9.2	Tokenisation, stop words
pandas	2.3.3	Manipulation des données tabulaires
matplotlib	3.10.8	Visualisations
NumPy	2.4.1	Calcul matriciel

Table 3.3 – Bibliothèques utilisées

Chapitre 4

Expérimentation

Ce chapitre présente le protocole expérimental mis en place, les résultats obtenus lors de la comparaison des classificateurs et de la sélection des features, ainsi qu'une analyse des performances observées.

4.1 Protocole expérimental

4.1.1 Partitionnement des données

Les 95 paires du corpus sont divisées en **80 % entraînement / 20 % test**, soit environ 76 paires pour l'entraînement et 19 pour le test. Le partitionnement est *stratifié* afin de préserver la distribution des quatre classes dans chaque sous-ensemble.

4.1.2 Métriques d'évaluation

Les modèles sont évalués à l'aide des métriques suivantes : accuracy, précision macro, rappel macro et F1-score macro. Le **F1 macro** est la métrique principale car, contrairement à l'accuracy, il traite chaque classe à égalité et pénalise les modèles qui ignoreraient les classes minoritaires (`cut`, `heavy`, `light`).

4.1.3 Déroulement des expériences

L'évaluation se déroule en trois étapes successives :

1. **Comparaison des classificateurs :** les cinq modèles sont entraînés et évalués avec les sept features complètes, afin d'établir une baseline et d'identifier les modèles les plus prometteurs ;
2. **Grille exhaustive features × modèles :** les $2^7 - 1 = 127$ sous-ensembles non vides de features sont croisés avec les 5 modèles, produisant **635 configurations** évaluées systématiquement ;
3. **Validation croisée :** une validation croisée stratifiée à 5 plis est appliquée à la meilleure configuration afin d'estimer la variabilité des performances.

4.2 Résultats des mesures de similarité

4.2.1 Comparaison des classificateurs sur toutes les features

La figure 4.1 compare les cinq modèles entraînés avec l'ensemble des sept features.

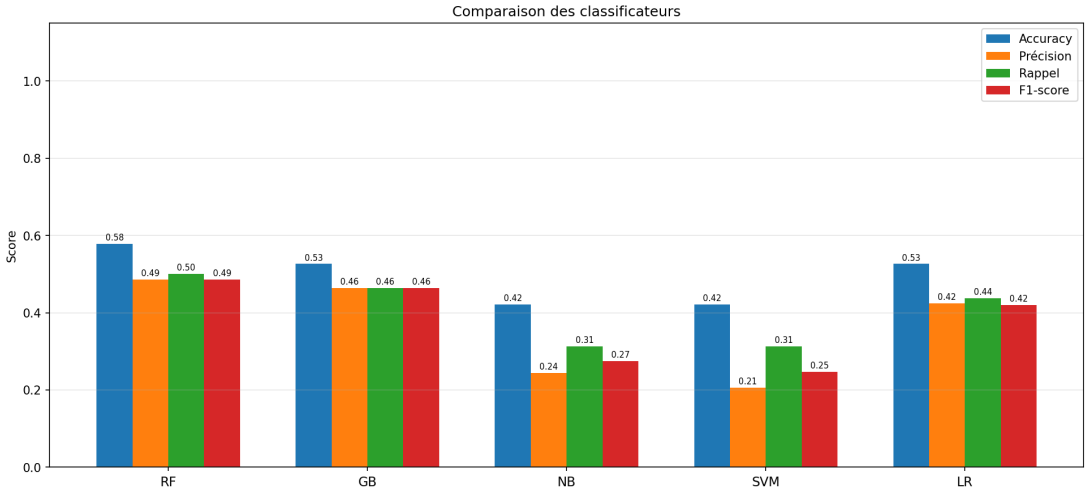

FIGURE 4.1 – Comparaison des cinq classificateurs avec les sept features

On observe que Random Forest et Naive Bayes obtiennent les meilleures performances dans cette configuration, tandis que la Régression logistique et le SVM peinent davantage sur ce corpus de taille réduite.

4.2.2 Pouvoir discriminant des features individuelles

Afin d'évaluer la contribution propre de chaque mesure, chaque feature est testée isolément. Les résultats montrent que `dice_sim` et `ner_jaccard` sont les plus discriminantes seules, avec un F1 macro atteignant respectivement 0.505 et 0.476 (avec Gradient Boosting). À l'inverse, `semantic_sim` et les features NER spécialisées (`ner_jaccard_person`, `ner_jaccard_org`) apportent peu d'information de manière isolée.

4.3 Performance des classificateurs

4.3.1 Grille exhaustive : 635 configurations

La figure 4.2 présente la distribution des F1 macro sur l'ensemble des 635 configurations testées.

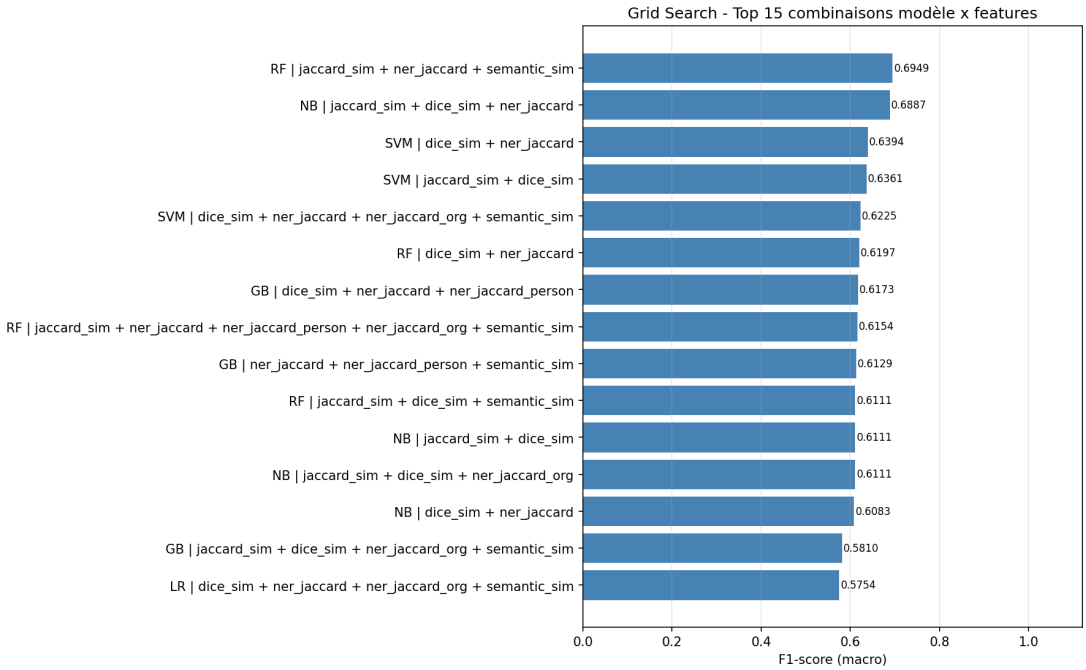

FIGURE 4.2 – F1 macro des 635 configurations (sous-ensembles de features × modèles)

La meilleure configuration identifiée est :

Random Forest avec {`jaccard_sim`, `ner_jaccard`, `semantic_sim`} ⇒ **F1 macro = 0.695**, accuracy = **73.7 %**

Le tableau 4.1 récapitule la meilleure configuration obtenue pour chaque modèle.

Modèle	**Meilleures features**	**F1 macro**	**Accuracy**
Random Forest	jaccard_sim + ner_jaccard + semantic_sim	**0.695**	**73.7 %**
Naive Bayes	jaccard_sim + dice_sim + ner_jaccard	0.689	73.7 %
SVM	dice_sim + ner_jaccard	0.639	68.4 %
Gradient Boosting	dice_sim + ner_jaccard + ner_jaccard_person	0.617	68.4 %
Régression log.	dice_sim + ner_jaccard + ner_jaccard_org + semantic_sim	0.575	63.2 %

TABLE 4.1 – Meilleure configuration par classificateur parmi les 635 testées

Un résultat notable ressort de cette analyse : utiliser l'ensemble des **7 features simultanément est contre-productif**. Le Random Forest avec toutes les features n'atteint qu'un F1 de 0.487 (accuracy 57.9 %), contre 0.695 avec seulement 3 features sélectionnées.

4.3.2 Étude d'ablation

La figure 4.3 présente les résultats de l'étude d'ablation conduite sur le meilleur modèle (Random Forest) : chacun des 127 sous-ensembles non vides de features y est évalué, ce qui

permet de visualiser l'impact de chaque combinaison sur le F1 macro.

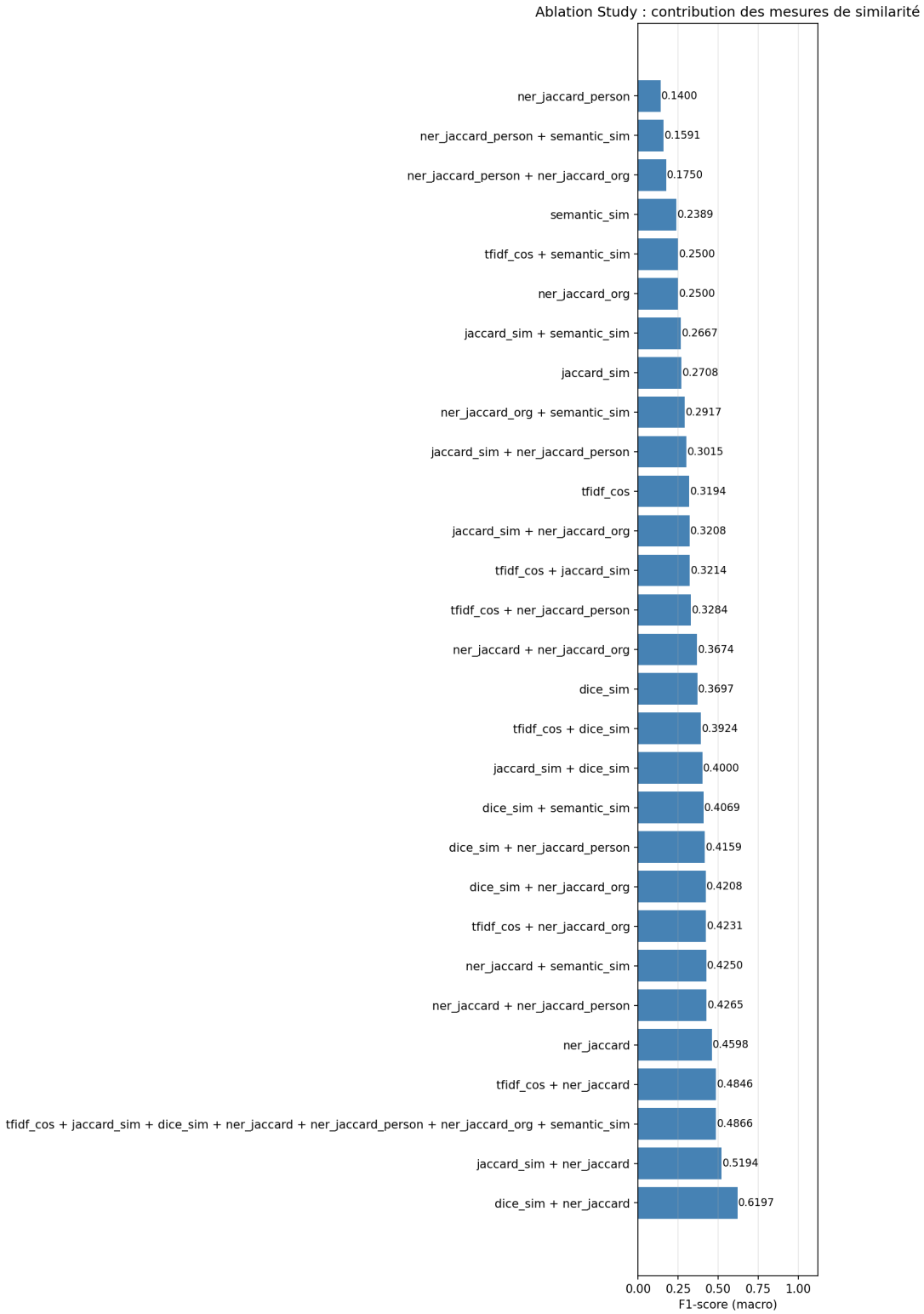

FIGURE 4.3 – Étude d'ablation : F1 macro selon les sous-ensembles de features (Random Forest)

On constate que les meilleures performances sont systématiquement obtenues avec **2 à 3 features**, confirmant qu'un sous-ensemble bien choisi surpasse l'utilisation de toutes les features.

4.3.3 Matrice de confusion du meilleur modèle

La figure 4.4 présente la matrice de confusion du Random Forest avec les features {`jaccard_sim`, `ner_jaccard`, `semantic_sim`}.

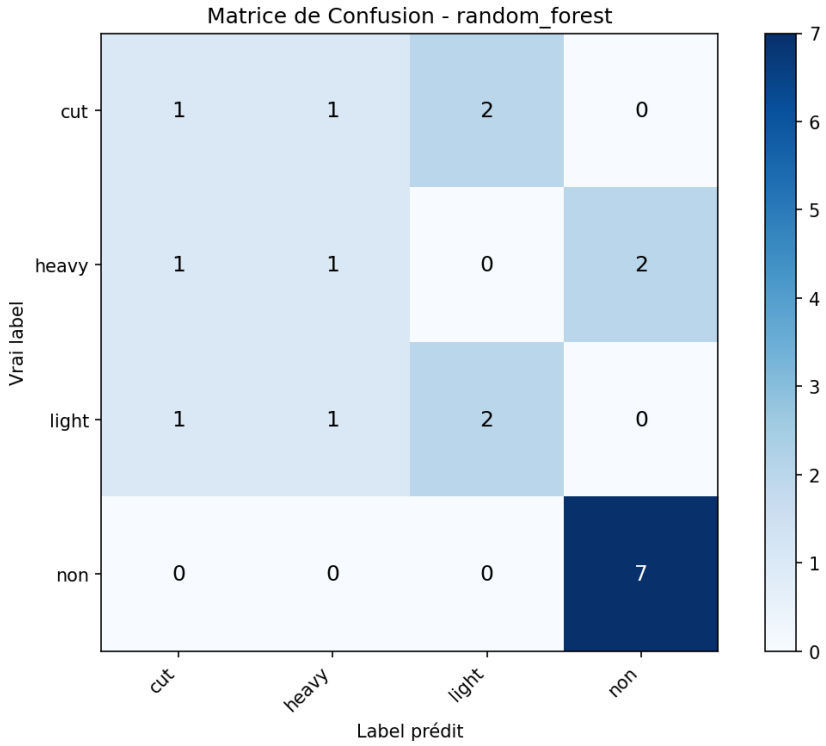

FIGURE 4.4 – Matrice de confusion du meilleur modèle (Random Forest, 3 features)

La classe `non` est bien reconnue, ce qui s'explique par sa surreprésentation dans le corpus (40 %). Les confusions les plus fréquentes concernent les classes `light` et `cut`, dont les frontières sont naturellement plus floues (paraphrase légère *vs* copie partielle).

4.3.4 Analyse SHAP

La figure 4.5 présente les valeurs SHAP du meilleur modèle, qui quantifient la contribution marginale de chaque feature à chaque prédiction.

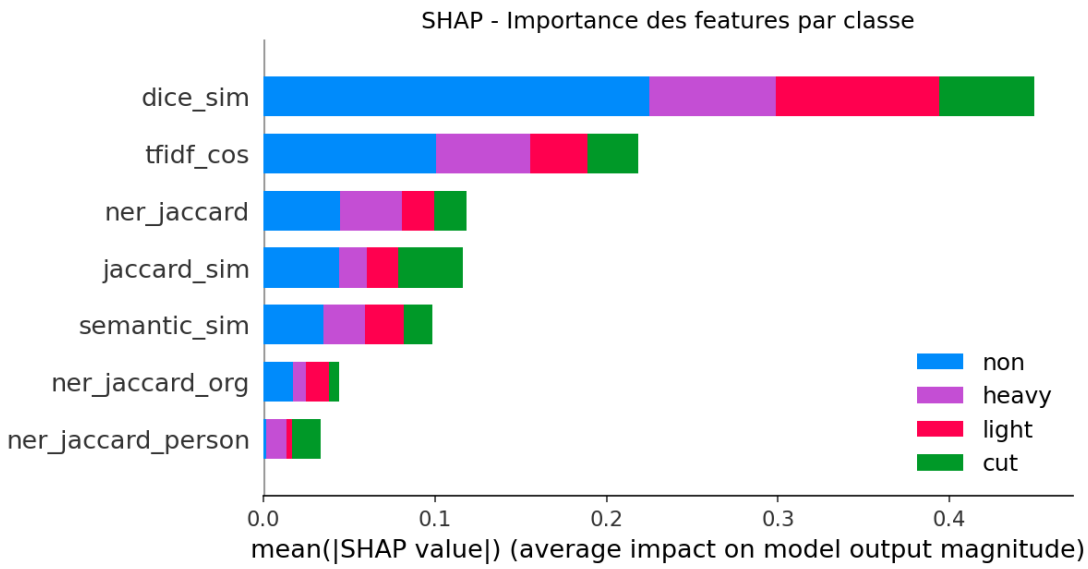

FIGURE 4.5 – Valeurs SHAP par feature et par classe (Random Forest, 3 features)

`jaccard_sim` est la feature la plus influente pour distinguer les cas de plagiat direct (`cut`, `heavy`), tandis que `semantic_sim` joue un rôle complémentaire pour les paraphrases (`light`).

4.4 Analyse et discussion des résultats

4.4.1 Complémentarité des features sélectionnées

La combinaison gagnante {`jaccard_sim`, `ner_jaccard`, `semantic_sim`} couvre trois niveaux d'analyse distincts :

— `jaccard_sim` (caractères uni-grammes) capte le chevauchement lexical brut, y compris pour les copies partielles (`cut`) ;

— `ner_jaccard` détecte la réutilisation d'entités nommées (noms, lieux, organisations), robuste aux reformulations de surface ;

— `semantic_sim` (Word2Vec) capte la proximité sémantique au-delà du chevauchement lexical exact, utile pour les paraphrases (`light`, `heavy`).

Cette complémentarité explique pourquoi ces trois features ensemble surpassent toute combinaison plus large.

4.4.2 Limites identifiées

Plusieurs limites doivent être soulignées :

— **Taille du corpus :** avec seulement 95 paires et 19 exemples de test, chaque erreur de classification représente 5,3 points d'accuracy. Les résultats sont à interpréter avec prudence, et la validation croisée reste indispensable pour confirmer leur robustesse ;

— **Word2Vec sur petits documents :** le modèle Word2Vec est entraîné sur seulement deux documents à la fois (quelques centaines de tokens), ce qui limite la qualité des embeddings et explique son faible pouvoir discriminant lorsqu'il est utilisé seul ;

— **Features NER spécialisées :** `ner_jaccard_person` et `ner_jaccard_org` sont souvent nulles sur des textes courts, ce qui les rend bruitées et pénalisantes lorsqu'elles sont ajoutées à de bonnes combinaisons.

Conclusion

La réalisation de ce projet sur la détection de plagiat et l'analyse d'identité d'auteurs a été une expérience enrichissante et stimulante pour notre équipe. À travers ce travail, nous avons pu concrétiser nos connaissances théoriques en traitement automatique du langage naturel et en apprentissage automatique, en mettant en pratique des concepts fondamentaux de représentation textuelle et de classification supervisée. En nous concentrant sur les objectifs définis, nous avons progressivement élaboré un système fonctionnel et performant, capable d'analyser des paires de documents et de détecter différents niveaux de plagiat (non/light/cut/heavy) avec précision.

Au fil de notre travail, nous avons également eu l'occasion de consolider nos compétences en matière de communication et de gestion de projet. La répartition efficace des tâches, les échanges réguliers et la résolution collective des problèmes techniques ont contribué à renforcer notre cohésion d'équipe et à maintenir un rythme de progression soutenu tout au long du semestre.

Bilan des contributions

Ce projet a permis de développer un système complet de détection de plagiat et d'analyse d'identité d'auteurs, entièrement implémenté en Python. Parmi les principales réalisations :

- Mise en œuvre d'une chaîne de traitement textuel robuste (prétraitement, extraction de 7+ caractéristiques de similarité : TF-IDF cosinus, Jaccard/Dice n-grammes, NER, Word2Vec) ;
- Implémentation de 5 classificateurs supervisés (RandomForest, SVM, NaiveBayes, etc.) avec comparaison systématique des performances ;
- Évaluation exhaustive sur un corpus réaliste (paires labellisées non/light/cut/heavy) démontrant l'efficacité des approches combinées ;
- Architecture modulaire réutilisable, testée via scripts dédiés et scripts d'évaluation automatisés.

Les résultats expérimentaux confirment que les mesures TF-IDF cosinus et Jaccard n-grammes sont les plus discriminantes, particulièrement pour détecter les plagiats "heavy" et "light".

Perspectives de travaux futurs

Pour améliorer le système, plusieurs extensions sont envisageables :

- Intégration de modèles transformers (BERT, Sentence-BERT) pour la similarité sémantique ;
- Analyse stylistique avancée (détection signatures auteurs via LSTM/attention) ;
- Extension au code source (détection plagiat programmation) ;
- Interface web pour utilisation par enseignants (upload documents, seuils configurables) ;
- Tests sur corpus multilingues et domaines variés (scientifique, journalistique).